Objective

Understand how unsanitized user input can lead to XSS vulnerabilities and how to prevent them.

Vulnerable Example

Create a file called `index.html`:<!DOCTYPE html>

```
<html>

<head>

  <title>XSS Demo</title>

</head>

<body>

  <h2>Leave a Comment</h2>



  <input type="text" id="comment" placeholder="Type a comment">

  <button onclick="addComment()">Post</button>



  <div id="comments"></div>



  <script>

      function addComment() {

          const comment = document.getElementById("comment").value;



          // Vulnerable code

          document.getElementById("comments").innerHTML +=

              "<p>" + comment + "</p>";
```

```
      }

  </script>

</body>

</html>
```

Testing the Vulnerability

Run the page locally and enter:

```
<b>Hello World</b>
```

Notice that the text is interpreted as HTML instead of plain text.

This demonstrates that user input is being inserted directly into the page without validation or sanitization.

Secure Version

Replace the vulnerable code with:

```
function addComment() {

  const comment = document.getElementById("comment").value;



  const p = document.createElement("p");

  p.textContent = comment;



  document.getElementById("comments").appendChild(p);

}
```

Why is this safer?

- `innerHTML` interprets content as HTML.

- `textContent` treats input as plain text.
- User input cannot be interpreted as markup.

What You Learn

This exercise teaches:

- Input validation
- Output encoding
- XSS prevention
- Secure DOM manipulation
- Why user input should never be trusted